

Hands-On-Learning: Select and Evaluate LLM Architectures for TechCorp

Learning Objectives

By the end of this activity, learners will be able to:

- Measure and compare latency, accuracy, and cost metrics across encoder-only, decoder-only, and encoder-decoder LLM families.
- Construct a decision matrix that aligns model characteristics with enterprise SLAs and budget constraints.



- Recommend mitigation strategies (prompt truncation, caching, retrieval) to satisfy strict latency requirements without sacrificing accuracy.



Description

In this hands-on practice, you will act as TechCorp's senior AI engineer and select the optimal LLM architecture for a dual-intent assistant serving retail customers. Your findings will inform the end-to-end architecture decision and benchmarking dossier, enabling you to build a defensible comparison matrix covering latency, cost, accuracy, and context handling, and then recommend an architecture ready for production planning.

Structure of the Activity

Dataset Information/Resources Required

You will work with TechCorp's anonymized customer intent logs and product metadata. The dataset includes labeled intents (product lookup, order status, returns), historical response times, and tokenized conversations capped at 1,024 tokens. Ensure you comply with TechCorp's privacy policy when storing derived metrics.

Dataset:

Jsonl dataset
(**m1_select_evaluate_llm_architectures_techcorp_intent_dataset.jsonl**)

Tools Used:

- Python 3.10
- Hugging Face Transformers 4.35+
- accelerate, evaluate, pandas, rich
- Jupyter Notebook (Lab environment)

Setup Requirements

- GPU access (CUDA or MPS) for latency profiling
- Access to starter code file: [m1_lab_starter.ipynb](#) in JupyterLab
- Install optional extras with: `pip install -U transformers accelerate evaluate datasets rich torch`
- Starter notebook bootstraps dataset loaders, evaluation shells, and profiling utilities

Scenario

TechCorp leadership needs quantitative evidence comparing LLM architecture families before allocating compute budget. As TechCorp's senior AI engineer, you must deliver a data-backed recommendation for a dual-intent assistant that services retail customers efficiently. The chosen architecture must meet strict latency SLAs (<200ms p95), stay under a \$5,000/month budget, and maintain high accuracy across product lookups, order status inquiries, and return processing.

Objective

Build a defensible comparison matrix covering latency, cost, accuracy, and context handling, then recommend an architecture ready for production planning.



Instructions for Participants

Activity 1: Baseline the Three Architectures

Objective

TechCorp leadership needs quantitative evidence comparing architecture families before allocating compute budget.



Dataset:

m1_select_evaluate_llm_architectures_techcorp_intent_dataset.jsonl - TechCorp intent dataset

Steps to Complete:

Step 1: Load the sampled TechCorp intent dataset and split into train/validation using the provided utility `prepare_data loaders()`.
Brief guidance: Review the dataset structure and ensure data loaders are configured for batch processing.

Step 2: Execute the baseline evaluation cell to gather accuracy and latency metrics for encoder-only (BERT-style), decoder-only (GPT-style), and encoder-decoder (T5-style) models.

Brief guidance: Use the following code pattern:

```
from metrics import LatencyProfiler, accuracy_report
profiler = LatencyProfiler(target_latency_ms=200)
encoder_results = profiler.evaluate_model(encoder_model, dataloaders['validation'])
decoder_results = profiler.evaluate_model(decoder_model, dataloaders['validation'])
encdec_results = profiler.evaluate_model(encdec_model, dataloaders['validation'])
```

Step 3: Populate the decision matrix template with metrics, qualitative notes, and initial risk indicators.

Brief guidance: Capture assumptions (hardware type, batch size) directly in the matrix so stakeholders can replicate findings.

Try It Yourself — Baseline Reporter:

```
def summarize_results(results_dict):    for family, stats in results_dict.items():
print(f"{family}: p95 latency={stats['p95_latency_ms']} ms |
accuracy={stats['accuracy']:.2%}") # < YOUR CODE HERE >: call
summarize_results with encoder_results, decoder_results, encdec_results
```

Expected Outcome: p95 latency breaching 200 ms should trigger red alert flags in the matrix.

Tips:

→ How does context length impact latency for each family under the same hardware constraints?

→ Which metrics (e.g., intent accuracy vs. recommendation coherence) should receive the highest weight?

→ Adjust batch sizes and observe latency shifts; experiment with CPU vs. GPU inference.

→ Run `tests/test_latency_thresholds.py` from the notebook to verify the profiler flags SLA breaches.

→ Expected output: All critical latency thresholds satisfied: False when any model exceeds 200 ms.

Activity 2: Optimize for SLA Compliance

Objective

TechCorp negotiates budgets quarterly; you must prove that optimization levers can keep spend under \$5,000/month.



Steps to Complete

Step 1: Apply quantization and caching strategies using the starter notebooks `optimize_pipeline()` helper.

Brief guidance: Use the following code pattern:

```
from optimizations import apply_int8_quantization, enable_response_cache
quantized_model = apply_int8_quantization(decoder_model)
cached_pipeline = enable_response_cache(quantized_model, cache_size=2048)
optimized_results = profiler.evaluate_model(cached_pipeline,
dataloaders['validation'])
```

Step 2: Re-run latency and cost projections, documenting results in the comparison matrix.

Brief guidance: Compare baseline vs. optimized performance.

Step 3: Draft a mitigation plan for context overflow using truncation, summarization, or retrieval augmentation.

Brief guidance: Document any trade-offs (e.g., loss in generation richness) so stakeholders understand mitigation side effects.

Try It Yourself — Cost Model Update:

```
from economics import estimate_monthly_cost
baseline_cost = estimate_monthly_cost(encoder_results) # < YOUR CODE HERE >: calculate
optimized_cost using optimized_results # < YOUR CODE HERE >: compute
savings percentage vs. baseline_cost
```

Expected Outcome: Optimized pipelines should reduce p95 latency below 200 ms and lower projected costs by at least 15%.

Tips:

→ Which optimization delivers the largest latency drop without harming accuracy?

→ How do caching strategies alter token consumption and monthly spend?

→ Swap in retrieval-augmented responses and measure the token delta.

→ Execute tests/test_cost_model.py to confirm monthly spend stays below \$5,000.

→ Expected output: Projected monthly spend: \$4,350 (example) with pass flag True.

Activity 3: Final Recommendation & Risks

Objective

TechCorp's launch review board demands a concise, data-backed recommendation with mitigation plans.



Steps to Complete:

Step 1: Complete the comparison matrix, highlighting latency compliance, accuracy, context tolerance, and cost.

Brief guidance: Ensure all data points are finalized and assumptions documented.

Step 2: Draft a one-page recommendation summarizing the chosen architecture, key metrics, and implementation roadmap.

Brief guidance: Use narrative storytelling—tie metrics to customer impact to win executive buy-in. Use the following code pattern:

```
from reporting import build_recommendation_brief  
brief =  
build_recommendation_brief(matrix, target_latency=200, target_budget=5000)  
print(brief)
```

Step 3: List open risks and monitoring metrics required for production readiness.

Brief guidance: Consider latency spikes, accuracy degradation, and budget overruns.

Try It Yourself — Risk Register Snippet:

```
risk_register = [ {"risk": "Latency spike during peak loads", "trigger": "p95 > 200ms", "mitigation": "Autoscale + adaptive batching", "owner": "Platform Ops"}, # < YOUR CODE HERE >: add two more risks tied to architecture and context handling ]
```

Expected Outcome: Recommendation brief clearly justifies the selected architecture with quantifiable evidence. 💡

Tips:

- What additional monitoring data will you collect beyond latency, accuracy, and cost?
- Which fallback strategies keep customer experience stable during rollbacks?
- Present alternative recommendations for future scaling phases (e.g., hybrid encoder-decoder).
- Run the final validation cell to ensure the recommendation brief includes SLA metrics and mitigation items.
- Expected output: Console log listing selected architecture, p95 latency, cost projection, and residual risks.

Compile Your Findings

1. Requirement 1: Decision Matrix

Deliverable: Decision matrix completed with quantitative latency, accuracy, and cost metrics.

Include:

- p95 latency measurements for all three architectures
- Accuracy scores across product lookup, order status, and returns intents
- Monthly cost projections
- Context handling capacity and limitations
- Documented assumptions (hardware, batch size)

2. Requirement 2: Optimization Experiments

Deliverable: Optimization experiments demonstrate the SLA compliance path.

Include:

- Quantization results with accuracy trade-off analysis
- Caching strategy impact on latency and cost
- Context overflow mitigation plan
- Cost savings calculation (baseline vs. optimized)

3. Requirement 3: Recommendation Brief

Deliverable: A recommendation brief articulates the chosen architecture, mitigations, and the monitoring plan.

Include:

- Selected architecture with justification
- Key performance metrics (latency, accuracy, cost)
- Implementation roadmap
- Risk register with triggers, mitigations, and owners
- Monitoring requirements for production

Program runs without errors

Summary

This activity provides hands-on experience with LLM architecture evaluation through the practical application of quantitative comparison frameworks. Participants have developed competency in measuring performance metrics, optimizing for enterprise constraints, and building data-driven recommendations, all of which are directly applicable to the deployment of production AI systems.

Key takeaways:

- Architectural evaluation must align with business SLAs.
- Building and interpreting quantitative comparison matrices.
- Optimization levers (quantization, caching, retrieval) enable cost-conscious performance.

Common Issues & Solutions

Problem: GPU not detected, leading to inaccurate latency readings.

Solution: Switch the runtime to GPU or use the provided cloud environment with CUDA enabled.

Problem: Token costs exceed the budget after optimizations.

Solution: Reduce max_tokens, enable summarization for longer contexts, and re-run the cost estimator.

Resources

Primary Materials:

→ **m1_lab_starter.ipynb** - Starter notebook with bootstrapped loaders and utilities

→ **m1_select_evaluate_llm_architectures_techcorp_intent_dataset.jsonl** - TechCorp intent dataset

Reference Documentation:

- Hugging Face Transformers readings
- Python profiling utilities documentation

Solution/Exemplar:

- **m1_lab_solution.ipynb** - Fully commented reference implementation

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.